

Simulação e Teste de Software (CC8550)

Laboratório 5: Teste de Unidade Avançado (FIRST, padrão AAA)

Prof. Luciano Rossi

Ciência da Computação
Centro Universitário FEI

2º Semestre de 2026

Laboratório 5

🎯 Objetivo

Objetivo desta aula

Implementar uma carteira digital e sua suíte de testes completa: padrão AAA, exceções customizadas, fixtures com setup/teardown, parametrização com IDs e uma suíte de síntese cobrindo transferência entre carteiras.

Como entregar

Ao final do laboratório, compacte todos os arquivos `.py` produzidos (código e testes) em um único `.zip` e envie pelo link desta atividade no **Moodle**.

Exercício 1

Exercício

Carteira digital: base

Implemente a classe `CarteiraDigital`:

- construtor recebe `saldo_inicial` (padrão 0)
- `depositar(valor)`: soma ao saldo
- `sacar(valor)`: subtrai do saldo

Escreva pelo menos 4 testes seguindo o padrão AAA (com os comentários # Arrange, # Act, # Assert), cobrindo saldo inicial, depósito e saque dentro do saldo disponível.

Exercício 2



Exercício

Saldo insuficiente

Crie a exceção `SaldoInsuficienteError` e ajuste `sacar` para levantá-la, com a mensagem "saldo insuficiente", quando o valor pedido for maior que o saldo. Escreva dois testes: um com `pytest.raises` que confira tipo e mensagem da exceção, e outro que confirme que o saldo **não muda** após a tentativa.

Exercício 3

🔗 Código

```
class CarteiraDigital:
    def __init__(
        self, saldo_inicial=0, log_path="carteira.log"
    ):
        self.saldo = saldo_inicial
        self.log_path = log_path

    def depositar(self, valor):
        self.saldo += valor
        with open(self.log_path, "a") as f:
            f.write(f"deposito:{valor}\n")
```

Exercício 3

Exercício

Arquivo de log limpo a cada teste

Cada depósito grava uma linha em `carteira.log`. Crie uma fixture `carteira_com_log` que, no **setup**, apague o arquivo se ele já existir, e no **teardown**, apague o arquivo criado pelo teste, mesmo se o teste falhar. Escreva um teste que use a fixture e confira o conteúdo do log após um depósito.

Exercício 4

Código

```
def classificar_transacao(valor):  
    if valor < 100:  
        return "pequena"  
    elif valor < 1000:  
        return "media"  
    else:  
        return "grande"
```

Exercício 4

Exercício

Classificação e o caso de fronteira

A especificação diz que 100 é **media** e 1000 é **grande** (limites inclusivos na categoria de cima). Escreva um teste parametrizado com pelo menos 6 casos, usando ids descritivos, cobrindo exatamente 100 e exatamente 1000. Um dos casos vai falhar: identifique por quê e corrija a função.

Exercício 5

⌕ Código

```
def transferir(origem, destino, valor):  
    origem.sacar(valor)  
    destino.depositar(valor)
```

Exercício 5

Exercício

Suíte completa de transferência

Escreva uma suíte de testes para a função acima:

- a) uma fixture que cria duas carteiras (origem com saldo, destino vazia)
- b) um teste parametrizado de transferência bem-sucedida, com pelo menos 3 valores e ids
- c) um teste que, ao tentar transferir mais que o saldo da origem, confirme que **nenhuma** das duas carteiras muda de saldo

Simulação e Teste de Software (CC8550)

Laboratório 5: Teste de Unidade Avançado (FIRST, padrão AAA)

Prof. Luciano Rossi

Ciência da Computação
Centro Universitário FEI

2º Semestre de 2026